

DevSecOps Pipeline Implementation

Student: Erik Heerke Witteveen

Student number: 413560

Course: DevSecOps (re-sit)

Assessor: Dr Mohsen Kakavand (KAMO)

Date: May 2026

What I built

I built a fully local DevSecOps pipeline running on my own machine through Docker. Every time I push to the main branch, the pipeline automatically runs security checks, builds the app, tests it, scans it for vulnerabilities and deploys it. No manual steps needed after the push.

The setup uses Forgejo as the git host and CI platform, and a local act_runner to run the pipeline jobs. Everything is defined in Docker Compose so it can be reproduced on any machine with Docker installed.

```
git push to main
--> Forgejo (localhost:3000)
    --> Gitea Actions triggers the pipeline
        --> Job 1: SAST scan (Semgrep)
        --> Job 2: Build, Test, SBOM, Vulnerability scan, Deploy
            --> App running at localhost:8080
```

Core Components

1. Self-hosted Forgejo Installation

I used Forgejo 9, which is a fork of Gitea and fully compatible with it. It runs locally through Docker Compose on port 3000.

I chose Docker Compose as the automation tool. The whole installation, including the volumes, network, health checks and environment settings, is written in `infrastructure/docker-compose.yml`. You can run `docker compose up -d` on any machine and get the exact same setup running in under a minute.

```
forgejo:
  image: codeberg.org/forgejo/forgejo:9
  container_name: forgejo
  environment:
    - FORGEJO__actions__ENABLED=true
    - FORGEJO__server__ROOT_URL=http://host.docker.internal:3000/
  ports:
    - "3000:3000"
    - "222:22"
```

Evidence: Forgejo is running at `http://localhost:3000` with the repository and Actions enabled. You can see this in the demo recording.

2. Pipeline Phases (Staged DSO Build Pipeline)

The pipeline is defined in `.gitea/workflows/pipeline.yml` and split into two jobs.

Job 1 runs the SAST security scan before anything gets built. If this fails, the build never starts. This is the "shift left" principle, catching issues as early as possible.

Job 2 runs after Job 1 passes. It does the build, tests, SBOM generation, vulnerability scan and deployment in one job. I kept them in one job so the built Docker image is available to all steps without needing a registry.

Stage | Tool | What it does

SAST | Semgrep | Scans source code before the build

Build | Docker | Compiles TypeScript and builds the image

Test | Jest | Runs automated unit tests

SBOM | Syft | Creates a full inventory of the image

Vulnerability scan | Trivy | Checks the image for known CVEs

Deploy | Docker | Replaces the running container

Evidence: Both jobs run green in Forgejo Actions on every push to main. This is shown in the demo recording.

3. Web Application Running and Hosted

I built a Node.js app in TypeScript. It has a real build step where TypeScript is compiled to JavaScript using `tsc` before it can run. The app is deployed at `http://localhost:8080`.

The app has three endpoints:

- `GET /` shows a dashboard with the version number, deploy time and uptime
- `GET /api/info` returns JSON with the app info
- `GET /health` is used by Docker for health checking

Evidence: The app is running at `http://localhost:8080`. The version number on the dashboard updates after every pipeline run.

4. Web Application Deployed by the Pipeline

Every push to main triggers a full pipeline run and ends with an automatic deployment. The version number on the app changes without me doing anything manually after the push.

```
- name: Deploy container
  run: |
    docker stop devsecops-app 2>/dev/null || true
    docker rm devsecops-app 2>/dev/null || true
    docker run -d \
      --name devsecops-app \
      --restart unless-stopped \
      -p 8080:3000 \
      devsecops-app:latest
```

Evidence: The demo recording shows the version number on the app changing after a push, without any manual steps.

Additional Concepts

5. Monitoring of the Infrastructure

I set up Uptime Kuma at `http://localhost:3001`. It monitors both the production app and the Forgejo instance, so I have visibility over the full stack.

Monitors configured:

- Production app: `http://host.docker.internal:8080/health`
- Forgejo: `http://host.docker.internal:3000/api/healthz`

Both check every 60 seconds and show response time graphs.

Evidence: Uptime Kuma shows both monitors green with response times and uptime history.

6. Monitoring of the Web Application

Uptime Kuma polls the `/health` endpoint of the app every 60 seconds. It records the response time and uptime percentage. If the app goes down it shows red straight away.

Evidence: The Uptime Kuma dashboard shows 100% uptime and around 20ms response time for the app monitor.

7. Automated SBOM (Software Bill of Materials)

I added Syft to the pipeline to generate an SBOM after every build. It creates a full list of every package and library in the Docker image in CycloneDX JSON format.

This is useful because if a vulnerability is found in a library later on, you can check the SBOM to see if your image is affected without having to rebuild it.

```
- name: Generate SBOM (CycloneDX)
  run: |
    syft scan ${env.IMAGE_NAME}:latest \
      -o cyclonedx-json \
      --file sbom.json
```

Evidence: The pipeline log shows the number of components catalogued after every build.

8. Immutable Infrastructure

Every build creates a new Docker image tagged with the Git commit SHA. The image is never changed after it is built. If a deployment needs to be rolled back, the previous SHA image can be started again directly.

```
docker build \
```

```
-t ${ env.IMAGE_NAME }:${ gitea.sha } \
-t ${ env.IMAGE_NAME }:latest \
./app
```

This means every deployment can be traced back to a specific commit and nothing is ever patched in place.

Evidence: The pipeline log shows both the SHA tag and the latest tag being created on every build.

10. Software Testing

I wrote a test suite using Jest and Supertest. The tests run in the pipeline against the built app and cover all three endpoints.

Tests included:

- /health returns HTTP 200
- /health returns {"status":"ok"}
- /api/info returns HTTP 200
- /api/info returns the correct app name
- /api/info returns a valid version number
- /api/info returns a healthy status

```
- name: Run tests
run: |
  cd app
  npm install --silent
  npm test
```

Evidence: The pipeline log shows all 6 tests passing on every run. If a test fails the pipeline stops before deploying.

12 and 13. Security Testing (SAST and Vulnerability Scanning)

SAST with Semgrep

Semgrep runs in Job 1 before the Docker build starts. It scans the TypeScript source code for security issues without running the code. This means a vulnerable commit never becomes a built image.

```
- name: Run Semgrep scan
run: |
  /tmp/semgrep-env/bin/semgrep \
  --config=auto \
  --json \
  --output=semgrep-report.json \
  ./app/src || true
```

Vulnerability scanning with Trivy

Trivy scans the built Docker image for HIGH and CRITICAL CVEs. It checks every package in the image including the base OS, not just the application code.

```
- name: Scan image with Trivy
```

```
run: |
  trivy image \
    --format table \
    --severity HIGH,CRITICAL \
    ${ env.IMAGE_NAME }:latest
```

Evidence: Every pipeline run logs the Semgrep findings and the full Trivy CVE table output.

14. Infrastructure Orchestration

The full stack (Forgejo, act_runner, Uptime Kuma) is defined in `infrastructure/docker-compose.yml`. Docker Compose handles service dependencies, persistent storage, networking and automatic restarts. The runner waits for Forgejo to pass its health check before starting.

You can reproduce the entire environment with one command:

```
docker compose up -d
```

Evidence: `infrastructure/docker-compose.yml` defines all three services and their configuration. The demo recording shows the stack starting from this file.

15. Automated Rollback

Before deploying, the pipeline saves the tag of the currently running image. After starting the new container it waits 20 seconds and checks the health status. If the container is unhealthy and there was a previous image, it stops the broken container, starts the old one again and exits with a failure so the pipeline shows as failed in Forgejo.

```
PREV_IMAGE=$(docker inspect --format='{{.Config.Image}}' devsecops-app 2>/dev/null || echo "")

sleep 20
HEALTH=$(docker inspect --format='{{.State.Health.Status}}' devsecops-app)

if [ "$HEALTH" = "unhealthy" ] && [ -n "$PREV_IMAGE" ]; then
  docker stop devsecops-app && docker rm devsecops-app
  docker run -d --name devsecops-app ... $PREV_IMAGE
  echo "Rollback complete. Previous version restored."
  exit 1
fi
```

Evidence: In normal operation the deploy step logs `Health status: healthy` and continues. The pipeline failure would be visible in Forgejo Actions if a rollback happened.

18. Own Concept: Docker Layer Caching

I added `--cache-from` to the Docker build step. Docker builds images in layers and when a layer has not changed since the last build it reuses the cached version instead of rebuilding it.

```
docker build \
  --cache-from ${ env.IMAGE_NAME }:latest \
  -t ${ env.IMAGE_NAME }:${ env.gitea.sha } \
  -t ${ env.IMAGE_NAME }:latest \
  ./app
```

In practice this means if I only change the TypeScript source file, the `npm install` layer is reused from the last build because `package.json` has not changed. Only the layers after the first changed file are rebuilt.

Evidence: On a push where only source files change, the pipeline log shows `CACHED` for the dependency layers.

Deployment Technology: Containerisation

The app is packaged as a Docker image using a multi-stage Dockerfile. Stage 1 installs all dependencies and compiles the TypeScript. Stage 2 starts fresh with just the compiled output and production dependencies. Dev tools and source files are not in the final image.

Security choices in the Dockerfile:

- `USER node` so the app does not run as root
 - `npm install --omit=dev` so no dev packages are in production
 - `HEALTHCHECK` so Docker restarts the container if the health endpoint stops responding
 - `--restart unless-stopped` so the container comes back after a reboot
-

Rubric Overview

| Concept | Done | Tool used

- 1 | Self-hosted Forgejo | Yes | Forgejo 9 via Docker Compose
 - 2 | Staged DSO pipeline (YAML) | Yes | Gitea Actions, 2 jobs
 - 3 | Web app running and hosted | Yes | Node.js/TypeScript on port 8080
 - 4 | Web app deployed by pipeline | Yes | Automatic on every push to main
 - 5 | Infrastructure monitoring | Yes | Uptime Kuma (Forgejo + app)
 - 6 | Web app monitoring | Yes | Uptime Kuma /health endpoint
 - 7 | Automated SBOM | Yes | Syft, CycloneDX JSON
 - 8 | Immutable infrastructure | Yes | Docker images tagged with SHA
 - 10 | Software testing | Yes | Jest + Supertest, 6 tests
 - 12 | Security testing (SAST) | Yes | Semgrep
 - 13 | Vulnerability scanning | Yes | Trivy, HIGH and CRITICAL
 - 14 | Infrastructure orchestration | Yes | Docker Compose
 - 15 | Automated rollback | Yes | Health check + image restore
 - 18 | Own concept | Yes | Docker layer caching
-

Repository Structure

devsec-project-erik/

```
■■■ .gitea/
■   ■■■ workflows/
■       ■■■ pipeline.yml           # pipeline definition
■■■ app/
■   ■■■ src/
■   ■   ■■■ index.ts               # Express app
■   ■   ■■■ index.test.ts         # Jest tests
■   ■■■ public/
■   ■   ■■■ index.html             # dashboard
■   ■■■ Dockerfile                 # multi-stage build
■   ■■■ package.json
■   ■■■ tsconfig.json
■■■ infrastructure/
■   ■■■ docker-compose.yml         # Forgejo, runner, Uptime Kuma
■   ■■■ runner-config.yaml
■   ■■■ .env.example
■■■ demo pipeline run.mov          # video demonstration
■■■ IMPLEMENTATION.md              # this document
■■■ SETUP.md                       # setup guide
■■■ README.md
```